Full Fine-Tuning

When LoRA Isn't Enough

FSDP · DeepSpeed ZeRO · Mixed Precision · Catastrophic Forgetting

WEEK 2 · DAY 10 **Wednesday, July 15 2026** **Day 10 of 84**

#	Topic	What You Learn
01	Decision Framework	Prompt vs RAG vs LoRA vs Full FT vs Pre-train
02	When Full FT Is Justified	5 cases: quality ceiling, distillation, alignment
03	Data Parallelism vs FSDP	DDP, ZeRO-0 through ZeRO-3, tensor parallelism
04	DeepSpeed ZeRO	Stage 1/2/3, offload, config JSON
05	Mixed Precision	BF16 vs FP16, gradient scaling, autocast
06	The Training Loop	Gradient clipping, gradient checkpointing, packing
07	Catastrophic Forgetting	Data mixing, NEFTune, weight decay prevention
08	Flash Attention 2	Tiled attention, 3-5x memory reduction
09	Cost Estimation	C=6ND calculator, 8B=$19, 70B=$170
10	FAANG in Production	Meta LLaMA · Google Gemma · Microsoft Phi · Amazon
11	Text-to-SQL Thread	Quality ladder, when to upgrade from LoRA
12	Architect's Lens	5-point checklist before committing to full FT

TEXT-TO-SQL CAPSTONE THREAD — Full Fine-Tuning defines the ceiling of Text-to-SQL quality. Today you learn when to go beyond LoRA, how much it costs, and the complete production pipeline: Full FT + DPO + Schema RAG targeting 96-97% execution accuracy.

Section 1 — The Definitive Decision Framework

You now know every major adaptation technique. This framework governs every production decision from today through Week 12:

```
STEP 1: Can better prompting solve this?
         YES -> Prompt engineer. Zero cost, instant. Always start here.

STEP 2: Does the model lack KNOWLEDGE (not skill)?
         YES -> RAG (Weeks 3-4). No training. Knowledge stays fresh.

STEP 3: Does the model lack a SKILL or BEHAVIOR?
         + Small dataset (<1K examples)  -> Few-shot prompting only
         + Medium dataset (1K - 50K)     -> LoRA (Day 9)
         + Large dataset (50K+)          -> LoRA r=64 OR Full Fine-Tuning
         + Alignment / safety change     -> Full FT + DPO (mandatory)
         + New architecture / dialect    -> Full Fine-Tuning only

STEP 4: Does the task require entirely new representations?
         YES -> Full pre-training ($10M+). Extremely rare.
```

> *This framework is the single most important mental model in AI Engineering. Most teams skip straight to fine-tuning when RAG would solve the problem in a day. Most LoRA users over-invest when prompting would close the gap. Always exhaust cheaper options before committing to training.*

Section 2 — When Full Fine-Tuning Is Justified

LoRA is the default. Full fine-tuning is the exception. Five cases where full FT is genuinely justified:

Case 1: The LoRA Quality Ceiling

LoRA trains 0.5-2% of parameters, reaching 90-95% of full FT quality. When the task demands 96-99%, full FT is required.

> *Example: SQL generation (Spider benchmark): LoRA r=64 = 91%, Full FT = 94%, Full FT + DPO = 96%*

Case 2: Deep Knowledge Injection

LoRA adapters lack capacity to learn fundamentally new vocabulary and reasoning patterns absent from pre-training data.

Example: Example: proprietary SQL dialect (StarRocks UDFs, custom aggregates) not present in LLaMA's 15T token pre-training corpus

Case 3: Alignment and Safety

LoRA fine-tuning for safety is shallow. For commercial deployment with strict safety requirements, DPO on top of full SFT is mandatory.

Example: Every frontier model (Claude, GPT-4, Gemini, LLaMA 3 Instruct) uses full SFT + DPO — none uses LoRA alone for safety alignment

Case 4: Distillation

Training a small model (7B) to match a large model (70B or GPT-4) requires full FT. The small model must reorganize all weights. LoRA cannot provide enough flexibility for teacher-student transfer.

Example: Microsoft Phi-3: 3.8B fully fine-tuned on GPT-4-generated data outperforms LLaMA 2 7B despite being half the size

Case 5: Building for Commercial Release

Publication quality requires state-of-the-art benchmark numbers. LoRA consistently underperforms full FT by 2-5% on held-out evaluations.

Example: Every published SOTA SQL model, code model, and medical model uses full fine-tuning

Section 3 — Distributed Training Strategies

Full fine-tuning on models 7B+ requires multiple GPUs. Three strategies distribute training across GPUs:

Data Parallelism (DDP) — The Simple Case

Copy the full model to every GPU. Split the batch across GPUs. Each GPU computes gradients independently, then gradients are averaged (All-Reduce) and weights updated. Limitation: every GPU must hold the full model — impossible for 70B on A100 80GB.

FSDP — Fully Sharded Data Parallel (Meta/PyTorch)

FSDP shards model weights, gradients, and optimizer states across all GPUs. No GPU holds the full model at any time. Before each forward pass, weights are gathered (All-Gather). After backward, gradients are scattered back.

```
For LLaMA 3 70B full fine-tuning with FSDP:

Model weights (FP16):          140 GB
Optimizer states (FP32 Adam): +560 GB
Total:                         700 GB

Sharded across 8x A100 80GB:  700 / 8 = 87.5 GB per GPU
+ activations (~15 GB):        ~102 GB per GPU
                                -> fits on A100 80GB (80 GB VRAM)

Minimum hardware for 70B full FT: 8x A100 80GB
```

DeepSpeed ZeRO — Configurable Sharding

DeepSpeed's ZeRO (Zero Redundancy Optimizer) provides four sharding stages. For LLaMA 3 70B across 8 A100 80GB GPUs:

Stage	What Is Sharded	Memory/GPU (70B)	Throughput
ZeRO-0	Nothing (vanilla DDP)	~700 GB (impossible)	Fastest
ZeRO-1	Optimizer states only	~280 GB (impossible)	Fast
ZeRO-2	Gradients + optimizer states	~175 GB (impossible)	Good
ZeRO-3	Weights + gradients + optimizer	~88 GB (fits!)	Moderate
ZeRO-3 + offload	ZeRO-3 + CPU RAM offload	~20 GB GPU	Slow

Tensor Parallelism — Splitting Individual Matrices

Splits individual weight matrices across GPUs. Each GPU holds a slice of every matrix. Requires NVLink (high-bandwidth interconnect). Used by Megatron-LM, NVIDIA, Google for Gemini. Not practical on standard cloud instances without NVLink.

Section 4 — Mixed Precision Training

BF16 — Preferred on A100/H100

BF16 keeps the same 8-exponent-bit range as FP32 (no overflow risk) but uses only 7 mantissa bits. No loss scaling needed. Native hardware support on A100/H100. Standard choice for full FT today.

```
from transformers import TrainingArguments

args = TrainingArguments(
    bf16=True,    # forward + backward pass in BF16
                  # optimizer states remain in FP32 (Adam requires precision)
                  # master weights kept in FP32, cast to BF16 for compute
)
```

FP16 with Gradient Scaling — Older GPUs

FP16 has max value ±65,504. Gradients can underflow to zero (vanishing) or overflow to NaN (exploding). Gradient scaling multiplies the loss by a large scale factor before backward, keeping gradients in FP16 range.

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()   # starts with scale_factor=2^16, auto-adjusts

for batch in dataloader:
    with autocast():                          # forward in FP16
        loss = model(**batch).loss
    scaler.scale(loss).backward()             # scaled backward pass
    scaler.unscale_(optimizer)                # restore true gradients
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    scaler.step(optimizer)                    # update weights
    scaler.update()                           # adjust scale factor
    # Scale doubled every N steps if no NaN; halved when NaN detected
```

Section 5 — The Full Training Loop

```
from transformers import AutoModelForCausalLM, TrainingArguments
from trl import SFTTrainer
import torch

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",  # 3-5x faster attention
    use_cache=False,                          # disable KV cache for training
```

```
)
model.enable_input_require_grads()            # required for gradient checkpointing

args = TrainingArguments(
    output_dir="./full-ft-output",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,     # effective batch = 128 on 8 GPUs
    learning_rate=2e-5,                # 10x lower than LoRA
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    bf16=True,
    gradient_checkpointing=True,       # recompute activations -> saves 60% memory
    max_grad_norm=1.0,                 # clip gradient norm to 1.0
    weight_decay=0.01,                 # L2 regularization against forgetting
    neftune_noise_alpha=5,             # noise in embedding layer -> better generalization
    fsdp="full_shard auto_wrap",       # ZeRO-3 equivalent
    fsdp_config={
        "fsdp_transformer_layer_cls_to_wrap": "LlamaDecoderLayer",
        "fsdp_backward_prefetch": "backward_pre",
        "fsdp_forward_prefetch": True,
        "fsdp_use_orig_params": True,
    },
    save_steps=200,
    save_total_limit=3,
    evaluation_strategy="steps",
    eval_steps=200,
    load_best_model_at_end=True,
    report_to="none",
)

trainer = SFTTrainer(
    model=model, args=args,
    train_dataset=train_ds, eval_dataset=eval_ds,
    tokenizer=tokenizer, max_seq_length=4096,
    dataset_text_field="text",
    packing=True,    # pack multiple short examples -> fewer steps, higher throughput
)
trainer.train()
```

Gradient Clipping — Non-Negotiable

Without gradient clipping, one bad batch can cause gradients to explode, corrupting the model. Clipping constrains the global gradient norm to max_norm=1.0 — standard for all LLM training.

```
# After backward(), before optimizer.step():
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# Algorithm:
# 1. Compute: global_norm = sqrt(sum(g_i^2 for all gradient tensors g_i))
# 2. If global_norm > 1.0: scale each gradient by (1.0 / global_norm)
```

```
# 3. Direction is preserved; only magnitude is capped
```

Gradient Checkpointing

The forward pass stores all intermediate activations for the backward pass. For a 4096-token sequence through 32 layers: ~40 GB of activation memory. Gradient checkpointing discards activations after each layer and recomputes them during the backward pass — trading 30-40% compute for 60% memory savings.

Section 6 — Catastrophic Forgetting

When you fine-tune a model on domain data, it can forget general capabilities. A SQL fine-tuned model might lose the ability to write coherent English prose. Four prevention strategies:

Strategy 1: Low Learning Rate

```
learning_rate=2e-5   # full FT standard (vs 2e-4 for LoRA)
# Tiny steps -> general knowledge erodes very slowly
# The model makes incremental adjustments rather than sharp turns
```

Strategy 2: Data Mixing (Replay)

```
from datasets import concatenate_datasets

# Mix domain data with general instruction data
train_data = concatenate_datasets([
    sql_dataset.select(range(45_000)),           # 90% domain-specific
    general_alpaca_dataset.select(range(5_000))  # 10% general instructions
])
# The 10% general data acts as an anchor — prevents overwriting general capability
# For critical deployments: 80/20 split is safer
```

Strategy 3: NEFTune — Noise in Embedding Layer

Add uniform noise to embedding vectors during training (not inference). Empirically improves instruction following and reduces overfitting. MT-Bench scores improved 15-25% vs standard fine-tuning in original paper.

```
args = TrainingArguments(
    neftune_noise_alpha=5,   # 5 is the standard value; try 10 for more regularization
    ...
)
# What it does: embedding = original_embedding + Uniform(-alpha/sqrt(L), alpha/sqrt(L))
# where L = sequence length
# Why it works: prevents memorization of exact training sequences
# -> better generalization to novel user queries
```

Strategy 4: Weight Decay (L2 Regularization)

```
weight_decay=0.01
# Adds penalty: L_total = L_task + 0.01 * sum(w_i^2)
# Penalizes large weight changes from pre-trained values
# Keeps weights close to their starting point -> less forgetting
```

ARCHITECT RULE: Always evaluate full FT models on BOTH domain tasks AND general benchmarks (MT-Bench, MMLU) before deploying. A model that gains 4% SQL accuracy but loses general question-answering ability is not production-ready. Data mixing is the most effective single countermeasure.

Section 7 — Flash Attention 2

Paper: Flash-Attention 2 (Dao et al., 2023). Standard attention computes the full N x N attention matrix (quadratic in sequence length). For a 4096-token sequence: 4096^2 x 4 bytes = 64 MB per layer, per example. This dominates GPU memory at long context lengths.

Flash Attention 2 restructures the computation using tiling — processing the attention matrix in blocks that fit in SRAM (fast on-chip memory). The full attention matrix is never materialized in HBM (slow off-chip memory).

```
Results vs standard attention (LLaMA 3 8B, 4096-token sequence):
  Standard attention:  64 MB / layer memory, 1.0x throughput
  Flash Attention 2:   ~12 MB / layer memory, 2-3x throughput

Installation and usage:
  pip install flash-attn --no-build-isolation  # requires Ampere+ (A100, H100, RTX 30/40xx)

  model = AutoModelForCausalLM.from_pretrained(
      "meta-llama/Llama-3.1-8B-Instruct",
      attn_implementation="flash_attention_2",
      torch_dtype=torch.bfloat16,
  )
  # One flag. All supported models (LLaMA, Mistral, Gemma) enable it automatically.
```

Flash Attention 2 is now standard in all production fine-tuning. LLaMA 3, Mistral, Gemma, Qwen all support it natively. Always enable it for any full FT run on sequences >2048 tokens. The memory savings often eliminate the need for an additional A100.

Section 8 — Cost Estimation Before You Commit

From Day 4: C = 6ND. Use this before every training run. Add FSDP communication overhead (efficiency ~60%):

```
def estimate_full_ft_cost(n_params, n_examples, avg_tokens,
                          n_epochs, gpu_tflops, n_gpus,
                          cost_per_gpu_hr, efficiency=0.60):
    d = n_examples * avg_tokens * n_epochs
    c = 6 * n_params * d                          # FLOPs
    gpu_secs = c / (gpu_tflops * 1e12 * efficiency)
    total_gpu_hrs = gpu_secs / 3600
    wall_hrs = total_gpu_hrs / n_gpus
    cost = total_gpu_hrs * cost_per_gpu_hr
    return f"GPU-hrs: {total_gpu_hrs:.1f} | Wall: {wall_hrs:.1f}h | Cost: ${cost:,.0f}"

# LLaMA 3 8B, 50K SQL examples, 3 epochs, 8x A100 @ $3/hr per GPU
estimate_full_ft_cost(8e9, 50_000, 512, 3, 312, 8, 3.0)
# -> GPU-hrs: 6.5 | Wall: 0.8h | Cost: $19

# LLaMA 3 70B, 50K examples
estimate_full_ft_cost(70e9, 50_000, 512, 3, 312, 8, 3.0)
# -> GPU-hrs: 56.9 | Wall: 7.1h | Cost: $171

# LLaMA 3 70B, 200K examples (production scale)
estimate_full_ft_cost(70e9, 200_000, 512, 3, 312, 8, 3.0)
# -> GPU-hrs: 227.4 | Wall: 28.4h | Cost: $682
```

Full FT vs LoRA — Side-by-Side Comparison

Dimension	LoRA r=16	LoRA r=64	Full FT
Trainable params	0.5%	2%	100%
Quality ceiling	90-95%	93-97%	98-100%
GPU memory (8B)	18 GB	20 GB	80 GB (8x A100)
Cost (8B, 50K examples)	$2	$3	$19
Training time	30 min	45 min	50 min
Forgetting risk	Low	Moderate	High (mitigable)
Adapter / output size	33 MB	130 MB	16 GB
Inference overhead	None*	None*	None

* After merge_and_unload()

Key insight: Full FT on 8B costs 10x more than LoRA r=16 but takes almost the same wall-clock time (bottleneck shifts from parameter count to data throughput). The quality gain of 3-7% on complex tasks usually justifies $17.

Section 9 — FAANG in Production

Meta — LLaMA 3 SFT + DPO (Published Recipe)

Meta's LLaMA 3 Instruct was created through full fine-tuning in two stages: SFT on ~10M high-quality instruction-response pairs (human-annotated + synthetic), followed by DPO using human preference data. They used FSDP across 16,384 H100s for the 405B model. The recipe is published — this is why LLaMA 3 Instruct is the base for most enterprise fine-tuning projects globally.

Google — Gemma Full FT on TPU

Google's Gemma instruction-tuned models are fully fine-tuned (not LoRA) on TPU v4 pods using JAX/T5X. TPUs provide native BF16 and INT8 matrix multiply in hardware — the same 8B model that costs $19 on A100 costs ~$8 on TPUs (3x higher throughput per dollar). Constitutional AI principles are baked into the DPO preference dataset for Gemini.

Microsoft — Phi-3 Distillation via Full FT

Phi-3-mini (3.8B) was created by full fine-tuning on 3.4 trillion tokens of GPT-4-generated synthetic textbook data. The distillation signal from GPT-4 required all 3.8B parameters to reorganize — LoRA's 0.5% capacity was insufficient. Phi-3-mini outperforms LLaMA 2 7B on reasoning benchmarks despite being half the size. The lesson: synthetic data quality + full FT beats scale.

Amazon — SageMaker Distributed Fine-Tuning

Amazon SageMaker ml.p4d.24xlarge instances (8x A100s, $32/hour) are the standard for full fine-tuning in AWS. SageMaker Training Jobs abstract FSDP/DeepSpeed setup: provide a HuggingFace training script, specify instance type and count, and SageMaker handles distributed launch, checkpointing to S3, and CloudWatch metrics. Amazon Titan models in Bedrock are fully fine-tuned on proprietary data using this infrastructure.

Netflix — Full FT for Deep Domain Shift

Netflix's content catalog spans 17,000+ titles across 50+ languages. When fine-tuning recommendation description generation models, the domain shift (proprietary metadata schema, internal genre taxonomy, brand voice guidelines across 30+ markets) is deep enough that LoRA consistently underperforms full FT by 4-6% on human evaluation. They run full FT quarterly with LoRA-based rapid updates between cycles.

Section 10 — Text-to-SQL Thread: The Quality Ladder

```
Text-to-SQL execution accuracy ladder (Spider benchmark):

1. Base LLaMA 3 70B, zero-shot:                ~40-50%
2. + Schema RAG (Week 3):                      ~60-70%
3. + LoRA r=16, 50K SQL examples:              ~85-88%
4. + LoRA r=64, 50K SQL examples:              ~89-91%
5. + Full Fine-Tuning, 50K SQL examples:       ~93-95%
6. + Full FT + DPO, 50K examples:              ~96-97%

When to move from step 4 to step 5:
  - Production SLA requires >92% execution accuracy
  - Model must learn proprietary SQL dialect (StarRocks, ClickHouse)
  - Dataset size >100K validated (question, SQL) pairs
  - Query complexity: CTEs, window functions, multi-way joins

Cost to reach step 5 for 8B model:
  Full FT, 50K examples, 3 epochs, 8x A100: ~$19 one-time
  Inference cost difference vs step 4: NONE (same model size after merge)
```

Full FT Config for SQL

```
# Full fine-tuning for SQL generation (8x A100 80GB):
args = TrainingArguments(
    learning_rate=2e-5,              # conservative to prevent forgetting
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,    # effective batch = 128
    bf16=True,
    gradient_checkpointing=True,
    attn_implementation="flash_attention_2",
    neftune_noise_alpha=5,           # prevent SQL pattern memorization
    weight_decay=0.01,
    max_grad_norm=1.0,
    fsdp="full_shard auto_wrap",
    # data mixing: 90% SQL examples + 10% general alpaca
    ...
)
# After training: evaluate on Spider dev set + proprietary schema test set
# Deploy: save -> AWQ quantize -> vLLM serve (Day 11)
```

> *PRODUCTION DECISION: For Agoda's Text-to-SQL use case (StarRocks + Snowflake + BigQuery + ClickHouse), the multi-dialect requirement and proprietary schema complexity push the recommendation toward Full FT + Schema RAG (Week 3) as the minimum viable architecture. LoRA r=64 is the fast-to-market path; Full FT is the production ceiling.*

Section 11 — Architect's Lens

```
FULL FINE-TUNING DECISION CHECKLIST

Before committing to full FT, answer all five:

[1] Have you proven LoRA is insufficient?
    Run LoRA r=64 first. Measure the quality gap on your eval set.
    If gap < 3%: LoRA r=64 is good enough. Save the money.
    If gap > 3%: Full FT is justified.

[2] Do you have enough high-quality data?
    < 10K examples:   Full FT will overfit. Use LoRA.
    10K - 50K:        Full FT viable with strong regularization.
    50K+:             Full FT is the right choice.
    Note: data quality > quantity. 10K validated pairs beats 100K noisy ones.

[3] Have you computed the cost?
    Use C = 6ND / (GPU_TFLOPS * efficiency).
    8B model, 50K examples: ~$19/run. Budget 5-10 runs: ~$100-200.
    70B model, 50K examples: ~$170/run. Budget 5-10 runs: ~$850-1,700.

[4] Do you have the infrastructure?
    8B full FT:  4x A100 80GB minimum (FSDP ZeRO-3)
    70B full FT: 8x A100 80GB minimum
    No multi-GPU: Use QLoRA r=64 instead.

[5] Have you planned for catastrophic forgetting?
    Add 10% general instruction data to training mix.
    Use: weight_decay=0.01, neftune_noise_alpha=5.
    Evaluate on BOTH domain task AND general benchmarks after training.
```

The Hidden Cost: Human Evaluation

Automated metrics (execution accuracy, perplexity) don't catch everything. Full FT models can pass automated evals but fail human evaluation — producing SQL that runs but returns wrong results, or generates plausible but semantically incorrect queries. Budget 20–40 human evaluation hours before declaring a full FT run production-ready.

Section 12 — Hands-On Exercises

Exercise 1: Cost Calculator

```
def estimate_full_ft_cost(n_params, n_examples, avg_tokens,
                          n_epochs, gpu_tflops, n_gpus,
                          cost_per_gpu_hr, efficiency=0.60):
    d = n_examples * avg_tokens * n_epochs
    c = 6 * n_params * d
    gpu_secs = c / (gpu_tflops * 1e12 * efficiency)
    total_gpu_hrs = gpu_secs / 3600
    wall_hrs = total_gpu_hrs / n_gpus
    cost = total_gpu_hrs * cost_per_gpu_hr
    print(f"GPU-hrs: {total_gpu_hrs:.1f} | Wall: {wall_hrs:.1f}h | Cost: ${cost:,.0f}")

print("--- LLaMA 3 8B, 10K examples, 8x A100 ---")
estimate_full_ft_cost(8e9,  10_000, 512, 3, 312, 8, 3.0)

print("--- LLaMA 3 8B, 50K examples, 8x A100 ---")
estimate_full_ft_cost(8e9,  50_000, 512, 3, 312, 8, 3.0)

print("--- LLaMA 3 70B, 50K examples, 8x A100 ---")
estimate_full_ft_cost(70e9, 50_000, 512, 3, 312, 8, 3.0)
```

Exercise 2: Identify the Right Approach

Scenario: SQL accuracy plateaued at 90% with LoRA r=64

Answer: Full FT — LoRA ceiling proven, gap > 3%, worth $19 investment

Scenario: Need knowledge of events after training cutoff

Answer: RAG — not a skill gap, knowledge problem. No training needed.

Scenario: Teach model a specific JSON output format

Answer: LoRA r=8 or even just prompting with structured output (Day 6)

Scenario: Publish open-source SQL model with SOTA numbers

Answer: Full FT + DPO — publication requires maximum quality

Scenario: 500 annotated customer support examples

Answer: LoRA r=8 with caution — borderline; high forgetting risk at this size

Scenario: Safety alignment for commercial deployment

Answer: Full FT + DPO mandatory — LoRA is insufficient for deep alignment

Summary — Day 10 Key Concepts

Concept	One-Line Definition
Full fine-tuning	Train all 100% of parameters; maximum quality; requires multi-GPU
FSDP	Shard weights+optimizer+gradients across GPUs; no GPU holds full model
ZeRO Stage 3	DeepSpeed equivalent of FSDP; configurable via JSON config
Data parallelism	Copy full model to each GPU; only viable when model fits one GPU
BF16 mixed precision	Train in BF16; no loss scaling; standard on A100/H100
Gradient clipping	Clip gradient norm to 1.0; prevents exploding gradients
Gradient checkpointing	Recompute activations in backward; saves 60% activation memory
Catastrophic forgetting	Full FT overwrites general skills; prevent with data mixing + low LR
NEFTune	Noise in embedding layer; improves generalization; alpha=5
Flash Attention 2	Tiled attention; 3-5x memory reduction; enable for sequences >2K
C = 6ND cost rule	8B + 50K examples = $19; 70B + 50K examples = $171

TOMORROW — Day 11: Model Serving

You have trained and fine-tuned the model. Now: how do you serve it at production throughput? Day 11 covers vLLM, TGI, continuous batching, PagedAttention, and throughput optimization — turning a fine-tuned model into a production API endpoint.

EAGLE EYE ADDENDUM — DAY 10: Pipeline Parallelism (PP) — Splitting Layers Across GPUs

Day 10 covered FSDP, DeepSpeed ZeRO, and Tensor Parallelism for distributed full fine-tuning. One distributed strategy was missing: Pipeline Parallelism — the third axis of parallelism used in large-scale training.

Pipeline Parallelism (PP)

Pipeline Parallelism divides the model's layers into sequential stages, assigning each stage to a different GPU. Data flows through GPUs in a pipeline: GPU 0 processes the first N layers, passes activations to GPU 1 (layers N+1 to 2N), and so on. Unlike Tensor Parallelism, which splits individual matrices, PP splits the model depth-wise — each GPU holds COMPLETE layers.

```
Model: 80 transformer layers total, 4 GPUs

Pipeline Parallelism assignment:
  GPU 0: Embedding + layers  1-20  (first quarter)
  GPU 1: Layers 21-40               (second quarter)
  GPU 2: Layers 41-60               (third quarter)
  GPU 3: Layers 61-80 + LM head     (final quarter)

Data flow (micro-batch 1):
  GPU 0: process micro-batch 1 (layers 1-20)  -> send activations to GPU 1
  GPU 1: receive from GPU 0                   -> process (layers 21-40) -> send to GPU 2
  GPU 2: receive from GPU 1                   -> process (layers 41-60) -> send to GPU 3
  GPU 3: receive from GPU 2                   -> process (layers 61-80) -> loss

While GPU 1 processes micro-batch 1, GPU 0 processes micro-batch 2
-> Pipeline keeps all GPUs busy (eliminates "pipeline bubble")
```

The Pipeline Bubble Problem

Without micro-batching, GPUs sit idle waiting for the previous stage to finish. This idle time is the 'pipeline bubble'. Micro-batching fills the pipeline: while GPU 3 finishes micro-batch 1, GPU 0 is already processing micro-batches 2, 3, 4. Bubble size = (n_stages - 1) / (n_stages + n_microbatches - 1). With 4 GPUs and 16 micro-batches: bubble = 3/19 = 16% overhead.

PP vs TP vs FSDP — The Three Axes

Strategy	What Is Split	Communication	NVLink Needed	Best For
Data Parallel (DP / FSDP)	Data (batch shards)	Gradient All-Reduce	No	Standard; scales to any GPU count
Tensor Parallel (TP)	Each weight matrix (columns/rows)	All-Reduce per layer	Yes (low latency req.)	Minimize latency; NVLink clusters

Pipeline Parallel (PP)	Model layers (depth-wise)	Activation point-to-point	No (P2P only)	Very large models; no NVLink

3D Parallelism: Combining All Three

Production training of the largest models (GPT-4, LLaMA 3 405B, Gemini Ultra) combines all three axes simultaneously — called 3D Parallelism or PTD-P:

```
3D Parallelism layout (example: 512 GPUs, 405B parameter model):

  Data Parallel groups:    64 replicas (each handling 1/64 of the data)
  Tensor Parallel groups:  8 GPUs per TP group (split each matrix 8-ways)
  Pipeline Parallel stages:1 GPU per stage (8 stages per TP group)

  Total GPUs: 64 (DP) x 8 (TP) x 1 (PP) = 512

Meta's LLaMA 3 405B training configuration:
  TP = 8  (NVLink within a node, 8x H100 per node)
  PP = 4  (point-to-point across nodes, InfiniBand)
  DP = 512 replicas
  Total: 8 x 4 x 512 = 16,384 H100 GPUs

Why PP instead of more TP for cross-node?
  NVLink: 900 GB/s bandwidth (within node, 8 GPUs)
  InfiniBand: 400 GB/s bandwidth (between nodes)
  TP requires very low latency all-reduce -> use NVLink only
  PP only needs point-to-point activation send -> works over InfiniBand
```

Pipeline Parallelism in HuggingFace / DeepSpeed

```
# DeepSpeed Pipeline Parallelism config:
ds_config = {
    "train_micro_batch_size_per_gpu": 2,
    "gradient_accumulation_steps": 4,
    "pipeline": {
        "stages": 4,          # number of pipeline stages = number of GPUs in PP group
        "activation_checkpoint_interval": 1,  # checkpoint every layer -> memory savings
    },
    "zero_optimization": {
        "stage": 1,           # ZeRO stage 1 within each PP stage
    },
}

# Launch with torchrun (4 GPUs, 4 PP stages):
# torchrun --nproc_per_node=4 train_pipeline.py --deepspeed ds_pp_config.json

# HuggingFace Accelerate with Pipeline Parallelism:
from accelerate import Accelerator
accelerator = Accelerator()
# Set ACCELERATE_PIPELINE_PARALLEL_SIZE=4 in config
# model layers distributed automatically across 4 GPUs
```

WHEN TO USE PP: Pipeline Parallelism shines when GPUs are connected via InfiniBand (cross-node) rather than NVLink (within-node), because PP only needs point-to-point activation transfers rather than all-reduce. For training on a cluster without NVLink: FSDP (ZeRO-3) + PP. For training within a single NVLink node: FSDP + TP. For the largest models (405B+): all three combined (3D Parallelism).